

Iteration 40 – Factory and Adapter Refinement

Resource Allocation System

Najib Attar

June 2026

1 Iteration Goal

The goal of Iteration 40 is to improve the internal design of the resource allocation system without adding a new allocation feature. This iteration focuses on two design-pattern refinements:

- adding `SpaceFactory` to centralize creation of concrete space objects;
- adding `BackendResultAdapter` to reduce direct Flask coupling to backend CSV result files.

The iteration improves separation of concerns, lowers coupling, and makes the project more consistent with the factory and adapter patterns already used in earlier iterations.

This iteration does not change allocation algorithms, rule behavior, request processing, input CSV schemas, or backend business logic.

2 Changes from Iteration 39

Iteration 39 introduced structured backend summary exports through `SummaryWriter`. It added generated output files such as:

- `data/request_summaries.csv`;
- `data/allocation_summaries.csv`.

However, two design issues remained:

- `DataLoader::loadSpaces(...)` still directly created concrete space subclasses;
- Flask still knew many backend CSV details and reconstructed UI summaries from several files.

Iteration 40 addresses these issues by introducing:

- `SpaceFactory` on the C++ backend side;
- `BackendResultAdapter` on the Flask web side.

3 Scenario

The system already creates several polymorphic objects using factories. For example, users are created by `UserFactory`, requests are created by `RequestFactory`, and allocation strategies are selected by `AllocationStrategyFactory`.

Before this iteration, spaces were different. `DataLoader` parsed `spaces.csv` and directly created `Classroom`, `Laboratory`, and `MeetingRoom` objects. This worked, but it coupled data loading to concrete space subclasses.

Similarly, the Flask dashboard could display backend results, but it had to read and interpret many backend CSV files directly. This made Flask route code aware of backend output details.

Iteration 40 improves both points while preserving current behavior.

4 Modules and Their Tasks

Module	Task
<code>SpaceFactory</code>	Creates concrete <code>Space</code> subclasses from a space type string.
<code>DataLoader</code>	Parses and validates CSV rows, then delegates space creation to <code>SpaceFactory</code> .
<code>Classroom</code>	Represents classroom spaces.
<code>Laboratory</code>	Represents laboratory spaces.
<code>MeetingRoom</code>	Represents meeting room spaces.
<code>BackendResultAdapter</code>	Adapts backend-generated CSV outputs into UI-ready structures for Flask routes.
<code>web/app.py</code>	Uses the adapter for allocation summary, request detail, schedules, and raw data access.
<code>ResourceAllocationSession</code>	Continues to coordinate backend loading, allocation, and export workflow.
<code>SummaryWriter</code>	Continues to export structured backend summary files.

5 Design Pattern Application

5.1 Factory Pattern and Creator

The first refinement is the addition of `SpaceFactory`. The project already uses factories for major polymorphic object families. Adding `SpaceFactory` makes space creation consistent with the existing factory-based design.

`SpaceFactory` creates:

- Classroom;
- Laboratory;
- MeetingRoom.

The factory is stateless and does not need to be a Singleton. It provides a simple creation method that receives the parsed space data and returns a `Space*`. If the space type is unknown, the factory returns `nullptr`. `DataLoader` then skips the row safely and prints a warning.

This follows the GRASP Creator principle because concrete object creation is moved into a dedicated creation component.

5.2 Adapter Pattern

The second refinement is the addition of `BackendResultAdapter` in the web layer.

The adapter does not compute allocations, evaluate rules, or generate suggestions. Those responsibilities remain in the C++ backend. Instead, the adapter reads backend-generated CSV outputs and converts them into UI-ready structures for Flask templates.

The adapter centralizes access to backend output files such as:

- request_summaries.csv;
- allocation_summaries.csv;
- request_results.csv;
- allocations.csv;
- users.csv;
- spaces.csv;
- request_participants.csv;
- user_busy_slots.csv.

This reduces the need for Flask route functions to know many backend CSV details. The route asks the adapter for allocation summaries, request details, schedules, or raw data, and then passes the returned structure to the template.

6 Design Principles Improved

Principle	How Iteration 40 Supports It
Separation of Concerns	<code>DataLoader</code> parses data while <code>SpaceFactory</code> creates spaces. Flask routes delegate backend-result interpretation to <code>BackendResultAdapter</code> .
Low Coupling	<code>DataLoader</code> no longer depends directly on concrete space subclasses. Flask routes are less coupled to backend output file relationships.
High Cohesion	<code>SpaceFactory</code> has one focused responsibility: creating spaces. <code>BackendResultAdapter</code> has one focused responsibility: adapting backend results for the UI.
Model-View Separation	Backend allocation behavior stays in C++. Flask remains a display and interaction layer.
Protected Variation	If new space types or new output presentation needs are added later, the changes can be localized more easily.
Design to Interface	Space creation returns the base <code>Space</code> type, allowing the rest of the system to work with the abstraction.

7 Iteration Comparison

Aspect	Iteration 39	Iteration 40
Space creation	<code>DataLoader</code> directly created <code>Classroom</code> , <code>Laboratory</code> , and <code>MeetingRoom</code> .	<code>DataLoader</code> delegates concrete space creation to <code>SpaceFactory</code> .
Factory consistency	Users, requests, and strategies used factories.	Spaces now also use a factory where polymorphic creation is justified.
Flask result access	Flask route/helper code interpreted several backend CSV files directly.	<code>BackendResultAdapter</code> centralizes backend result access for important routes.
Backend behavior	Allocation algorithms, rules, and strategies worked through existing backend services.	Backend behavior remains unchanged.
Output files	Normal and structured output files were exported.	Output files remain unchanged and are adapted for UI display.

8 Updated Class Diagram

Figure 1 shows the updated class diagram for Iteration 40. The diagram should include the new `SpaceFactory` and `BackendResultAdapter` components.

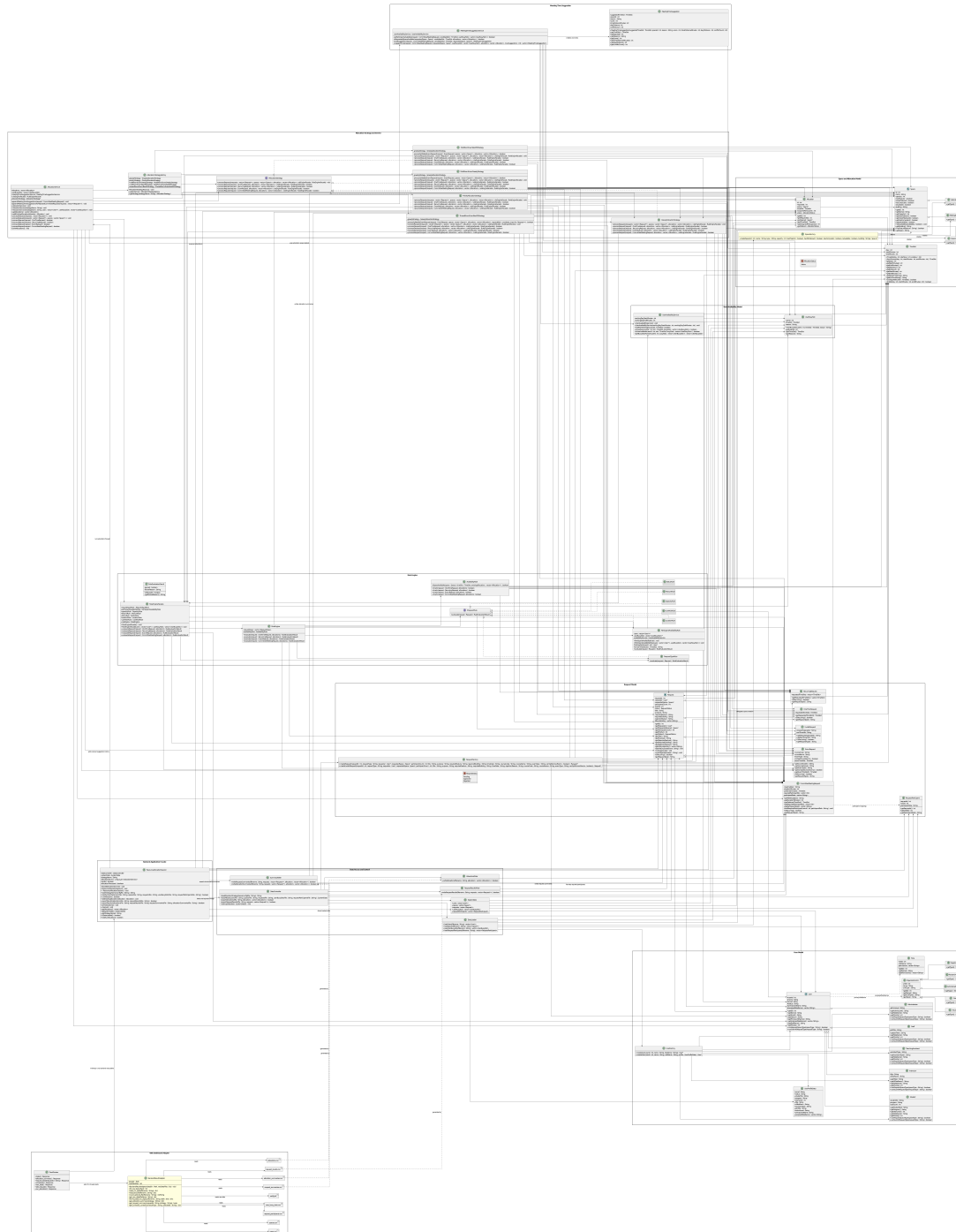


Figure 1: Updated class diagram after Iteration 40

The expected class relationships include:

- `DataLoader` uses `SpaceFactory`;

- `SpaceFactory` creates `Space` objects;
- `Space` is the base class of `Classroom`, `Laboratory`, and `MeetingRoom`;
- Flask routes use `BackendResultAdapter`;
- `BackendResultAdapter` reads backend output CSV files;
- `ResourceAllocationSession` and `SummaryWriter` continue to generate backend outputs.

9 Updated Workflow Diagram

Figure 2 shows the updated workflow after Iteration 40.

5. The Flask route asks `BackendResultAdapter` for UI-ready data.
6. The adapter reads backend output files and prepares the result structure.
7. The Flask route passes the data to the template.
8. The template renders the page.

10 Prototype Implementation

10.1 SpaceFactory

The new backend factory is implemented using:

- `src/models/SpaceFactory.h`;
- `src/models/SpaceFactory.cpp`.

The factory receives the parsed fields from `spaces.csv`, including ID, name, type, capacity, feature flags, availability, and building. It then creates the correct concrete subclass.

`DataLoader::loadSpaces(...)` now performs parsing, validation, duplicate-ID checking, and warning output. It does not directly instantiate concrete space classes. This keeps `DataLoader` focused on loading data and makes concrete space creation easier to extend later.

10.2 BackendResultAdapter

The new web-side adapter is implemented using:

- `web/backend_result_adapter.py`.

The adapter prepares UI-ready data for routes such as:

- `/allocation-summary`;
- `/request/<id>`;
- `/schedules`;
- `/raw-data`.

This makes the route functions thinner. The route no longer needs to directly know all details of how backend result files are joined together to create a display card or schedule grid.

11 Testing and Verification

The latest completed backend verification result for the repository reported:

Verification item	Result
Backend/unit/integration tests	79 passed, 0 failed
Backend smoke test	Passed
SpaceFactory creation tests	Passed
DataLoader delegation test	Passed
Schedule consistency tests	Passed

The main backend test command is:

```
powershell -ExecutionPolicy Bypass -File tests\run_tests.ps1
```

For the web adapter change, Flask smoke checks were performed for important routes. The checked routes returned safe HTTP responses, including:

- /;
- /allocation-summary;
- /schedules;
- /schedules?type=user&id=2;
- /schedules?type=space&id=301;
- /request/1;
- /request/40;
- /request/999999;
- /raw-data?file=request_summaries.csv;
- /raw-data?file=allocation_summaries.csv;
- /raw-data?file=not_allowed.csv;
- /add-request?type=CommitteeMeeting.

These checks confirm that invalid raw-data file names are handled safely, missing request detail pages do not crash, schedules still render, and committee meeting request creation remains available.

12 What Did Not Change

Iteration 40 is a design refinement, not a new allocation feature. The following behavior remains unchanged:

- allocation algorithms;
- rule evaluation behavior;
- allocation strategy behavior;
- request and user factory behavior;
- input CSV schemas;
- backend output CSV schemas;
- committee meeting validation;
- exam sharing and multi-room exam behavior;
- backend responsibility for allocation decisions.

No backend business logic was moved into Flask. Flask still displays backend-generated results and does not compute approvals, rejections, suggestions, or allocations.

13 Remaining Limitations

The adapter improves separation, but it does not remove all frontend/backend coupling. Some UI behavior still depends on CSV output files because the project intentionally uses CSV as the integration boundary.

Remaining limitations include:

- meeting suggestions are still exported through lifecycle-history text rather than a separate structured suggestion file;
- Flask still writes request input CSV rows for the demo workflow;
- future frontends may benefit from a JSON or API-oriented result boundary;
- some older helper functions may remain in `web/app.py` for compatibility during the gradual refactor.

These limitations are acceptable for this iteration because the goal was a small, safe adapter-pattern improvement, not a full web architecture rewrite.

14 Summary

Iteration 40 improves the design of the resource allocation system by applying patterns only where they solve real problems.

SpaceFactory completes factory-style creation for major polymorphic object families. It reduces coupling between **DataLoader** and concrete space subclasses while preserving the existing CSV format and loading behavior.

BackendResultAdapter introduces an adapter layer between Flask routes and backend-generated result files. It reduces direct route-level knowledge of backend output relationships and supports better frontend replaceability.

Together, these refinements improve separation of concerns, low coupling, high cohesion, model-view separation, and protected variation. The system behavior remains stable, and the changes prepare the project for future UI or API improvements without changing backend allocation logic.